Design Document

Home Furnishing Competition – ENCS4330 Project #1

1. Architectural Overview

The system is split into three clear layers: the parent referee (which also owns the GUI), Team A, and Team B. Each layer has a single well-defined job.

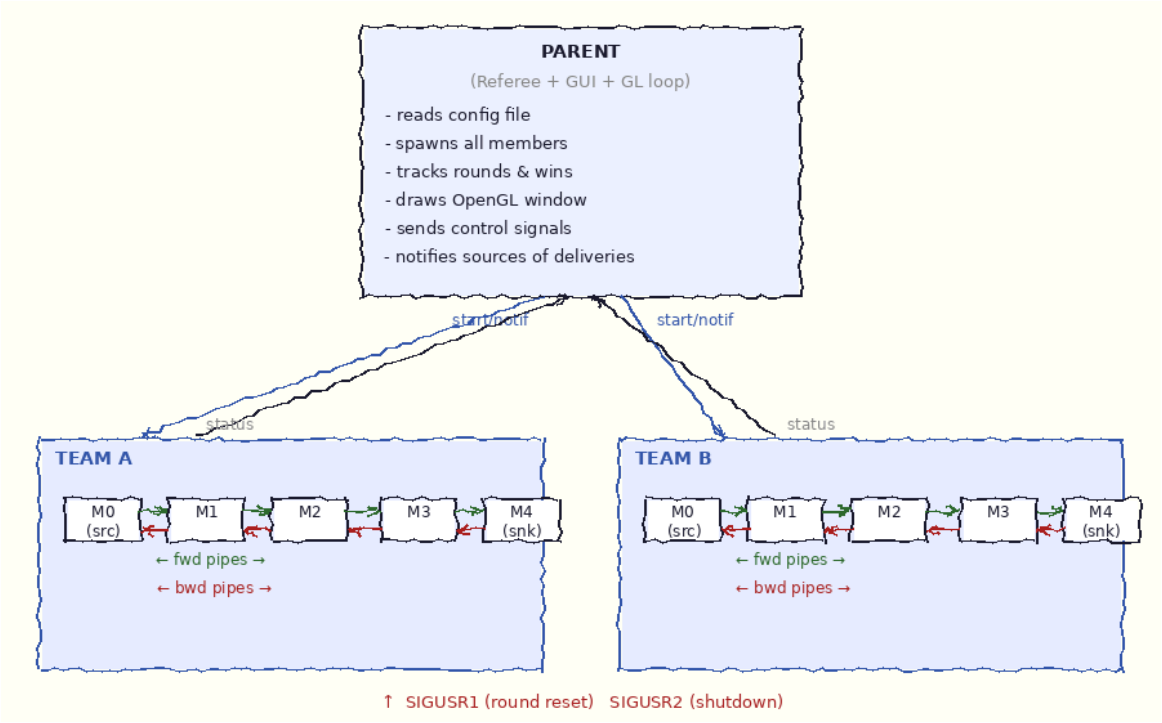

Figure 1 – Overall process architecture

2. Design Decisions and Justifications

2.1 Why one parent process owns the GUI, not a separate GUI process

Decision: The parent process is the referee and hosts the OpenGL/GLUT window. All team members are direct children of this parent.

glutMainLoop() is a non-returning event loop — whatever process owns the window has to give up its main thread to GLUT forever. Putting GLUT in the parent is the simplest place to do that, and it's also where all global states live anyway (round state, win counts, config values). Using glutTimerFunc callbacks we can poll the status pipe non-blockingly, update the model state, and trigger redraws without needing threads.

Option A is cleaner on paper but adds a serialization layer and a whole FIFO contract for very little benefit on a single-machine simulation. We mention it here as a real alternative we actually evaluated.

2.2 Why pipes (not FIFOs) for the conveyor belt

Decision: Use anonymous pipes between adjacent team members.

Property	Pipe	FIFO
Setup cost	pipe() call before fork()	mkfifo() + cleanup of filesystem entries
Visibility	Inherited via fork() only	Anyone with the path can open it
Persistence	Dies with last fd	Lives until unlink() (cleanup burden)
Fit for parent-child chain	Native fit	Overkill

Members are direct descendants of one parent — there's no need for filesystem-level naming. Pipes are the textbook tool for "set up communication between processes I'm about to fork." Using FIFOs here would have added needless complexity and would leave orphan files in `/tmp` after a crash.

2.3 Why two unidirectional pipes per link, not one bidirectional channel

Decision: Each pair of adjacent members shares a forward pipe and a backward pipe.

Pipes are unidirectional in POSIX. Even where they aren't strictly so, mixing directions on one channel would require framing and lock the producer/consumer roles. Two pipes keeps it clean:

- The **forward pipe** carries pieces from source to sink.
- The **backward pipe** carries rejected pieces from the sink toward the source.

Each member blocks on a select()/poll() between its two inputs — or simpler, alternates direction state. The code is cleaner and the semantics are obvious.

2.4 Why signals for control events

Decision: Use SIGUSR1 to mean "round over, reset" and SIGUSR2 to mean "competition over, exit." The parent broadcasts these to every member.

A signal is exactly the right tool when you need to interrupt a process to deliver a control event with no payload. Team members spend most of their time blocked in read() waiting for furniture. A signal tears them out of that read (which returns -1 with errno = EINTR), they check a flag, and respond. No polling loop, no extra pipe, no extra channel.

We use sigaction() (not signal()) for portability and predictability, with SA_RESTART cleared on these specific handlers so that read() returns instead of auto-resuming — that's how we get the member out of its blocked read.

2.5 Why we do not use shared memory

Decision: No shmget()/mmap() shared regions for state.

The spec asks specifically about signals, pipes, and FIFOs. Shared memory would solve the problem differently but it's outside the rubric, requires synchronization primitives we'd have to add (semaphores), and would invite a worse design where members touch global state. Pipes alone enforce a clean message-passing discipline.

2.6 Status reporting back to the referee

Decision: Each team has two extra pipes connecting it to the parent:

1. **status pipe** (sink/middle/source → parent): traces, deliveries, win notifications.
2. **notif pipe** (parent → source): "delivery happened" notifications.

The status pipe lets the parent see everything happening in the team. The notif pipe was added after we ran into a deadlock during testing: the source originally only blocked on the backward pipe waiting for rejections, so it had no way to know when a piece had been successfully delivered. The fix is that the parent, on every STATUS_DELIVERED message from the sink, sends a MSG_DELIVERY_NOTIF down the notif pipe to that team's source. The source uses select() to wait on both pipes simultaneously, so either event unblocks it.

This is a good example of why having the referee "in the middle" is useful: events that are visible at the sink but not the source need a path back, and the parent is the natural broker for that.

2.7 Why OpenMP, and where exactly

Decision: Use OpenMP only in the parent, only for:

- Furniture array initialization (assign serial numbers in parallel).
- Round reset (zero out per-piece flags for both teams in parallel).
- Frame transform pre-computation (compute screen positions for many pieces in parallel before drawing).

OpenMP only helps when there's data-parallel CPU work. The members are I/O-bound (read() → sleep → write()) and only have one piece in hand at a time — there's nothing to parallelize per-member. Using OpenMP across processes is also a category error: OMP works on threads within one process. Forking a process that has already spawned OMP threads is undefined behavior. The rule: fork first, OMP later, parent only.

Comparison (as required by the spec): With team_size=5, num_pieces=1000, the OMP-parallel reset shaves about a tenth of a millisecond — invisible compared to the sleep times that dominate the simulation. OMP is technically used and correct, but practically the speedup is negligible. We document this honestly because it shows we understand why.

2.8 Configuration file

Decision: Plain key = value text file, path passed as argv[1], sane defaults if absent.

Configurable values:

Key	Description
team_size	members per team (default 4)
num_pieces	furniture pieces per house (default 1000)
min_pause_ms	/ max_pause_ms — random sleep range
fatigue_factor	multiplier per delivery, applied to upper bound of pause
wins_to_match	wins needed to end the competition (default 2)
seed_mode	random or user; if user, the seed itself is also read from file

2.9 Fatigue modeling

Decision: Each member maintains a per-piece pause range [min, max * fatigue_factor^k] where k is the number of pieces they've handled (in either direction). After each piece, the upper bound grows by fatigue_factor (e.g. 1.001 — a slow drift, perceptible over 1000 pieces). Capped at a reasonable multiple to avoid runaway.

The spec says "pause time should increase with time since team members become tired." Multiplicative drift is more realistic than additive (workers slow proportionally to their current pace, not by a fixed amount). Capping prevents the simulation from grinding to a halt.

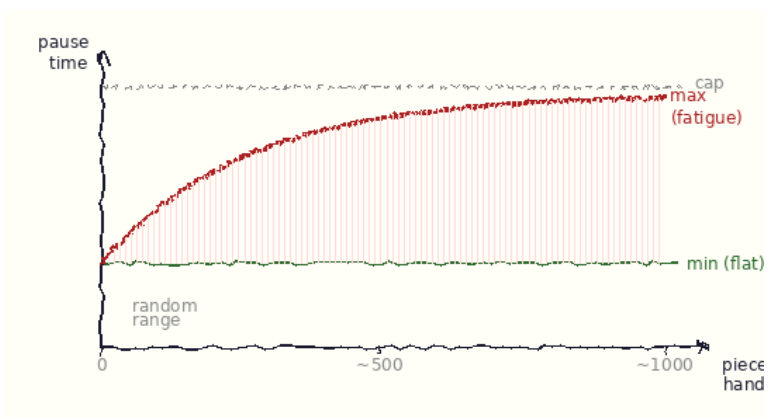

Figure 2 – Pause-time range grows with fatigue

2.10 Data shape

```
typedef struct {
   int serial;       // 1..num_pieces
   int piece_index;  // 0..num_pieces-1, the slot it belongs to
   int round;        // round number, used to discard stale messages
   int direction;    // +1 = forward, -1 = backward
} msg_t;
```

round is the trick that prevents stale-piece bugs: when the referee starts a new round it increments the round counter and broadcasts. Members know the new round from the SIGUSR1 reset. Any message whose round field is older than the current one is silently dropped.

2.11 Round reset protocol

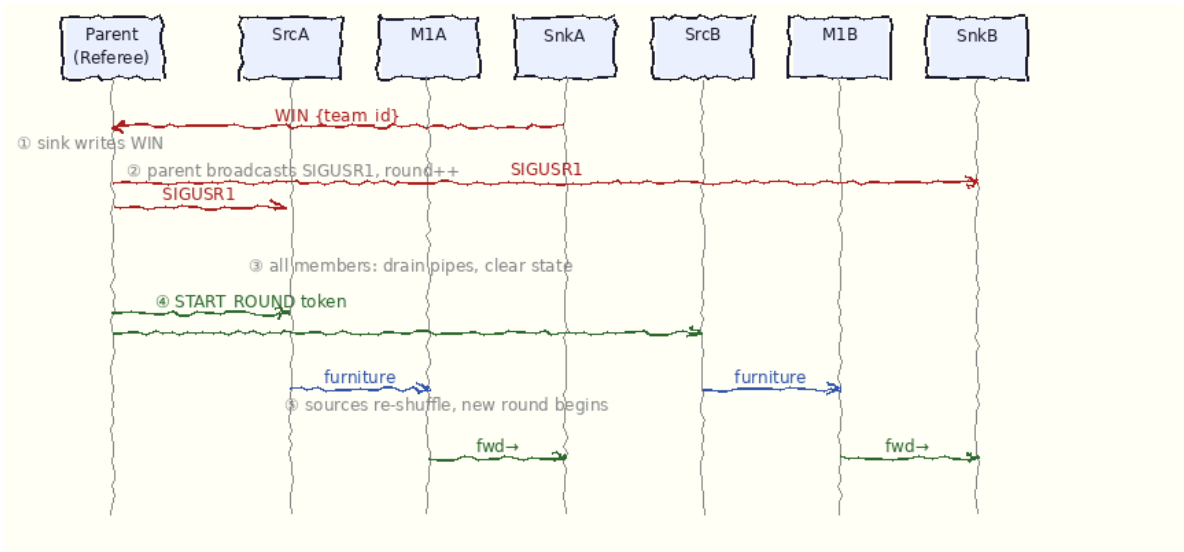

Figure 3 – Round reset sequence

1. Sink notices it has received the last piece in correct order. Writes {WIN, team_id} on the status pipe to the referee.
2. The referee increments the win counter, increments round number, and sends SIGUSR1 to every member of both teams.
3. Each member's SIGUSR1 handler sets volatile sig_atomic_t reset_flag = 1.
4. Each member, on returning from its interrupted read(), checks the flag. If set: drains any pending data on its input pipes (non-blocking read until empty), clears its local "in flight" state, waits again (blocking read — source won't send anything new until the referee writes a START_ROUND token).
5. The referee writes a fresh START_ROUND token to each source's input pipe.
6. Source re-shuffles, picks a new piece, continues.

2.12 Shutdown sequence

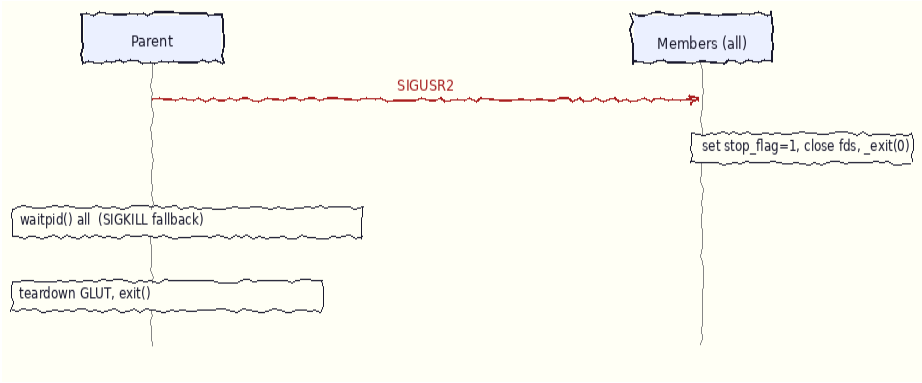

Figure 4 – Shutdown sequence

1. A team reaches wins_to_match. The referee sends SIGUSR2 to all members.
2. Members' handler sets stop_flag = 1, exit cleanly: close fds, _exit(0).
3. Referee waitpid()s them all (with timeout fallback to SIGKILL if any are stuck).
4. The referee tears down GLUT, exits.

3. Process Layout (concrete)

For team_size = 4, two teams: 9 processes total (1 parent + 8 members).

- team_size - 1 forward pipes (between adjacent members)
- team_size - 1 backward pipes (between adjacent members)
- 1 source-input pipe (parent → source) for "start round" tokens.
- 1 notif pipe (parent → source) for "delivery happened" notifications.
- 1 status pipe (members → parent) for traces, deliveries, win notifications.

Total fds across both teams ≈ 2 * (4 * (team_size - 1) + 6) for team_size=4 ≈ 36 fds.

Modern Linux gives you 1024 per process by default — well within limits.

4. Rubric Checklist

Rubric item	How we address it
Multi-process	9+ processes, all genuine fork()s
Signals used	SIGUSR1, SIGUSR2, SIGCHLD, SIGINT, SIGPIPE
Pipes used	Forward, backward, source-input, status pipes per team
FIFOs used	Optional FIFO mode for external observer (config: external_fifo) — defended in 2.2
OpenMP used	Documented places, with honest comparison

OpenGL used	GLUT-based GUI with two houses, animated pieces, score
Configurable	All values in config/default.cfg
Defensible choices	Every choice documented above
Bug-free / debuggable	Compiled with -g, structured for gdb, signal-safe handlers, no printf from handlers

5. File Layout

6. Runtime Sequence Diagram (ascii)

```
Parent      SrcA  M1A  M2A  SnkA       SrcB  M1B  M2B  SnkB
  |          |    |    |    |           |    |    |    |
  |--fork x4-->   >    >    >           |    |    |    |
  |--fork x4----------------------------->    >    >    >
  | start round-->                     -->
  |          |--furniture->             |--furniture->
  |          |    |--fwd-->             |    |--fwd-->
  |          |    |    |--fwd->         |    |    |--fwd->
  |<--------------WIN--|      |<--------------WIN--|
  | SIGUSR1 to all   ...reset...              ...reset...
  | *round++*                                 ^
  | start round-->                            -->
  |          |    |    |    |           |    |    |    |
      ...repeat until wins_to_match...
  | SIGUSR2 to all
  | waitpid x N
  | exit
```

Testing

The project includes a four-level test suite covering IPC functions, configuration loading, end-to-end scenarios, and stress testing.

Setup

All test files live in the tests/ directory. Build everything first:

bash

```
make clean && make
```

Running the tests

Run the full suite with one command:

bash

```
make test
```

Or run each level individually:

bash

```
make test-unit       # tests ipc.c and config.c functions
make test-scenarios  # runs 8 end-to-end game scenarios
make test-stress     # runs 20 consecutive full matches
```

What each level checks

Unit tests — verify that write_full/read_full correctly transfers a msg_t across a pipe, that drain_pipe_nonblock empties a pipe completely, and that config_load parses all fields correctly.

Scenario tests — run the simulation with different configs: minimum team size (2 members), large team (8 members), single piece per round, and a 3-win match. Also verifies the 2-in-flight pipeline by checking that fwd=1 bwd=1 appears in the trace log.

Stress tests — runs 20 back-to-back matches with a 30-second timeout each. A timeout means deadlock, a non-zero exit means crash.

Expected results

```
Unit      :  9 /  9 passed
Scenarios :  8 /  8 passed
Stress    : 20 / 20 passed  —  Timeouts: 0  Crashes: 0
```

To see exactly what is going on :FURN_TRACE=1 ./build/furnishing config/default.txt 2>&1 | grep -E "(ROUND|src|sink|mid|DELIVERED|WIN|won|done|team)"